

C Programming: Lecture I

FYD401



Credits

- The slides for the course were prepared by Andreas Heinz (Chalmers University of Technology)
 - Minor modifications were made
-

Overview of Lecture 1

- Computers and programming
- Programming in general
- C: history, development and properties
- C: basics
 - Hello World!
 - Structure of a C program
 - Simple I/O

Vägen till C
Chapters 1 to 3

We live in a society absolutely dependent on science and technology and yet have cleverly arranged things so that almost no one understands science and technology. That's a clear prescription for disaster.

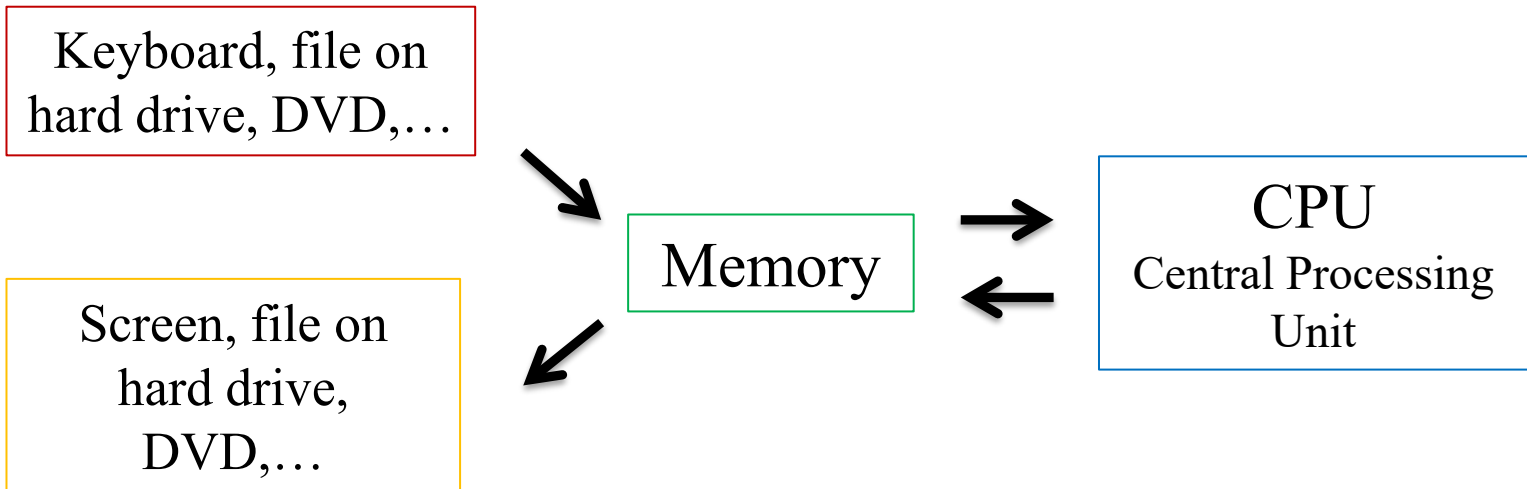
Carl Sagan

Computers

What is a Computer?

Definition of a computer

- A device which can manipulate data according to the instructions of a program.
- Early: mechanical, then wires, relays and transistors
- Now: integrated circuits
- von Neumann architecture
 - program/instructions and data are both stored in the memory



Types of Computers

- Supercomputer / Computer Cluster
 - Many processors, multi-core processors
 - High-performance parallel computing
 - Personal computer
 - PC, Mac, ...
 - Embedded systems
 - cars, refrigerators, data acquisition systems, ...
-

Computer Programming (I)

- Computers are digital devices
 - 0 and 1 only → on/off
 - Computers have a limited set of simple operations
 - CPUs work using Boolean logic operations
 - Computer programs are needed to perform complex tasks by using the simple logic operations
 - Programs are the interface between humans and computers
-

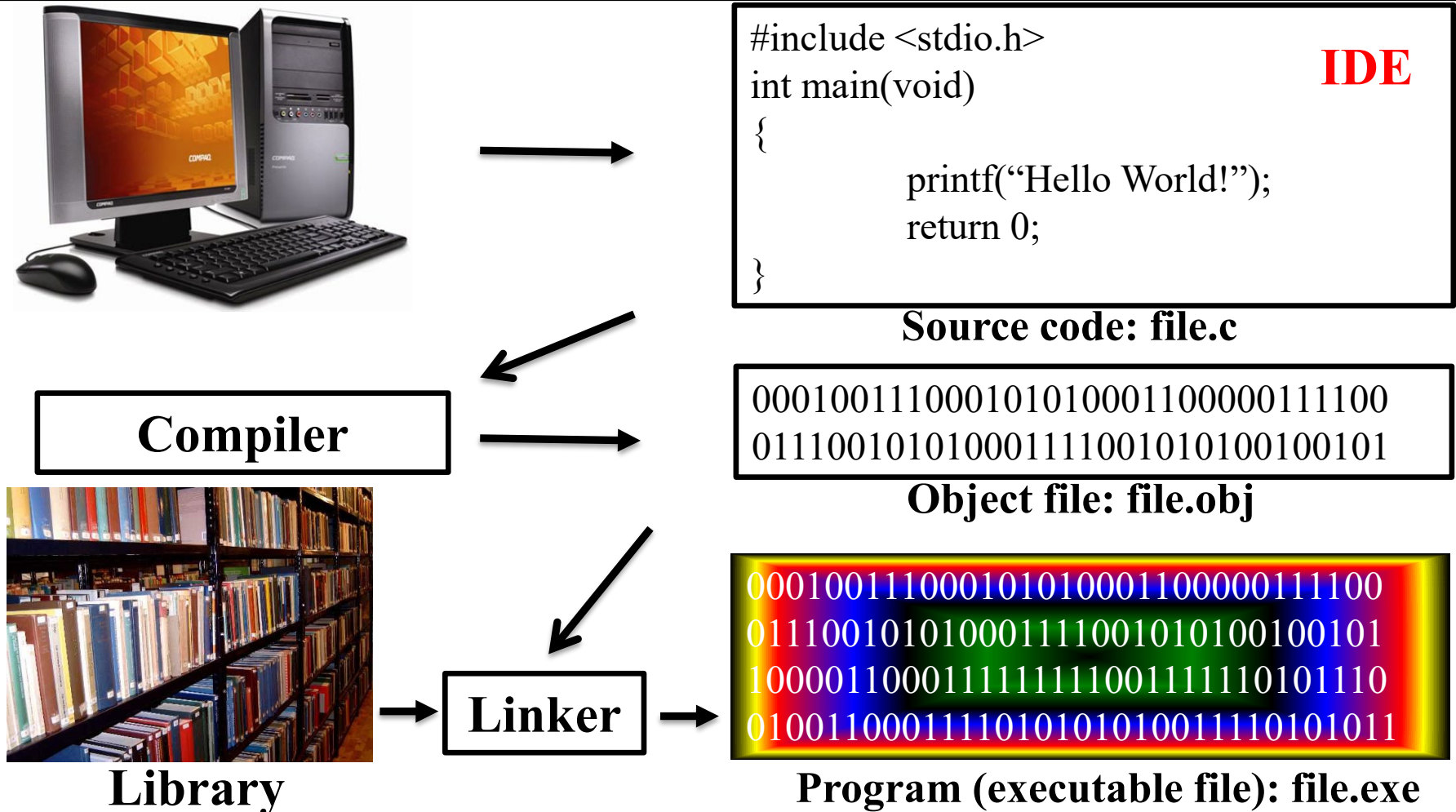
Computer Programming (II)

- Many different programming languages
 - Example: C
 - Programming languages allow us to translate our intention into commands the computer hardware can understand
 - We can do a lot of programming without knowledge of the computer hardware
-

Computer Programming (III)

- Computer program
 - Set of instructions which allows to use the computer for a task; typically to manipulate data
 - Platform: hardware + operating system
 - (e.g. PC + Windows)
 - Integrated Developer Environment (IDE)
 - This is where we can type in the code for the program
-

How to Create a Program?

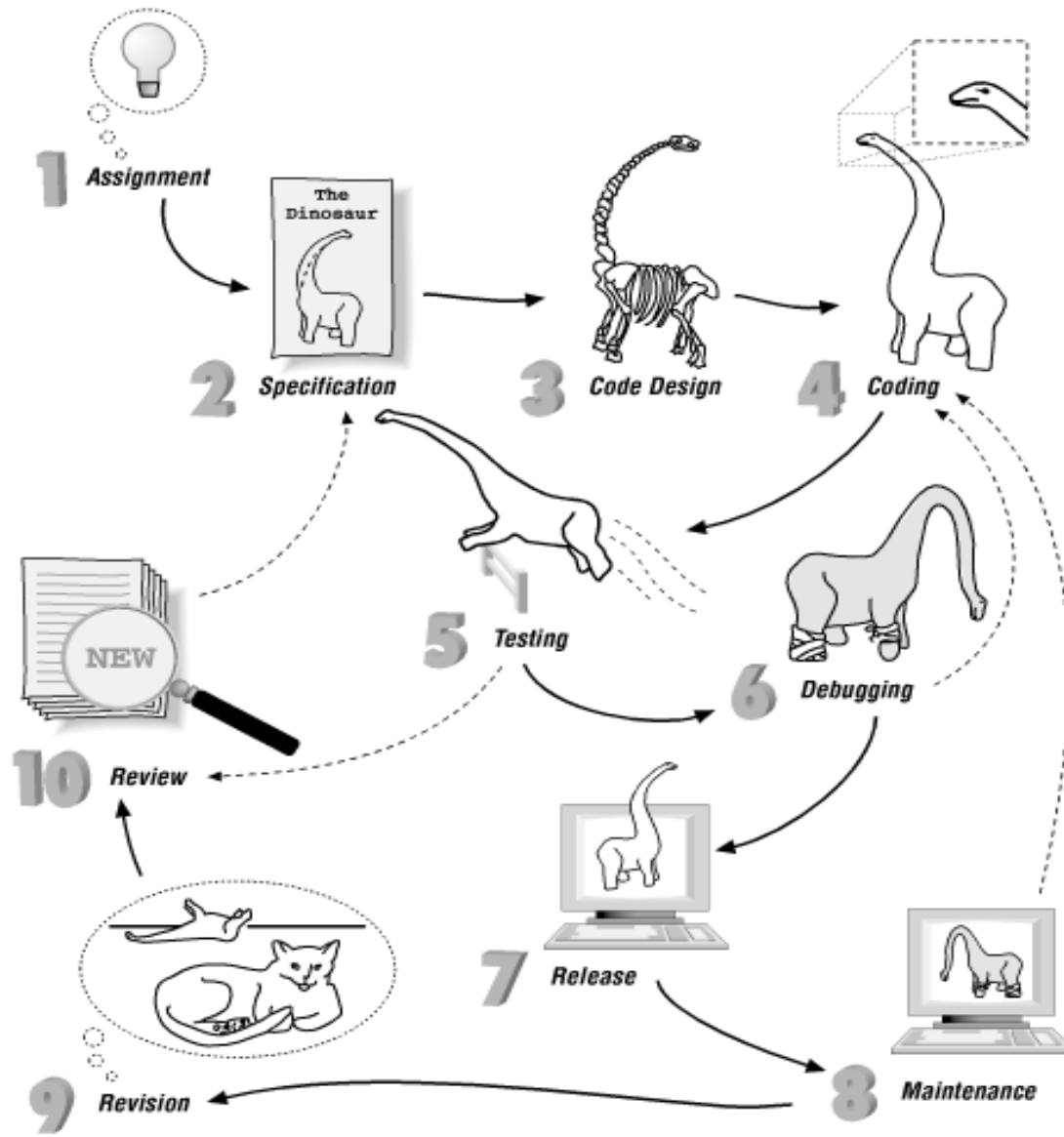


Programming and Planning

The problem: what, how, for whom, when?

1. Define the problem.
2. Outline the solution.
 - > flow chart
3. Choose/design an algorithm.
 - > model of the program
 - > a bare set of instructions
4. Select a language, compiler, integrated developer environment,... (does not change for every program)
5. Convert the algorithm into a program.
6. Check the program!





Life Cycle of a Program

More realistic picture of programming.

from "Practical C Programming" by S. Oualline

Programming

- Pay attention to details!
 - Computers are not forgiving, and C compilers are (intentionally) far from being foolproof
- “Computer errors” are usually human errors
 - They are made by either the user or the programmer (or both)
- Most important rule:



C programming language

- Why C?
 - Widely distributed language that allows for fast programs
 - mid- / low-level language
 - Small number of built-in functions
- Why not C?
 - Not safe (unless you work for it)
 - Old: it does not include, e.g., object-oriented programming
 - Learn C++, JAVA or C# directly

There is no programming language, no matter how structured, that will prevent programmers from writing bad programs.

L. Fon

History and Development of C

- Predecessors
 - Algol 60 -> BCPL -> B ->(NB).. C ... ca. 1972 Bell Labs (Dennis Ritchie)
 - Took OS UNIX from assembler to a "higher" programming language (1973).
 - Goal:
 - Simplify programming (higher level of abstraction)
 - Separate out the architecture-dependent parts of UNIX
 - Standards
 - ANSI C (1989) (American National Standards Institute), ISO C 99
 - C99
 - C11, C18
 - Development
 - Additional features while maintaining backwards compatibility
 - New architecture – C widely distributed
 - New languages "based on" C: C++ (pure extension), C#, Java, ...
-

Hello World! (I)

```
/* Create a source code for a program that prints ...*/  
/* The filename may be, e.g., hello.c */  
  
#include <stdio.h>  
  
int main (void)      // int main () also works  
{  
    printf ("Hello\n World!\n");    /* that's correct */  
}
```

To standard output: **Hello
World!**

Note: C is case sensitive!

Hello World! (II)

```
/* printing program */  
#include <stdio.h>  
  
int main (void) // int main () also works  
{  
    printf ("Hello\n World!\n"); /* that's correct */  
    return 0;  
}  
  
    // classic example
```

Comments: **// comment is the rest of the line (in C99)**
 or comment is enclosed by /* */

- **no nested comments**
 - **very important for documentation, readability, and you passing this course!**
-

Hello World! (III)

```
/* printing program */
```

```
#include <stdio.h>
```

```
int main (void) // int main () also works
```

```
{
```

```
    printf ("Hello\n World!\n");    /* that's correct */  
    return 0;
```

```
}
```

```
// classic example
```

Pre-processor directive:

- start with “#”
- no “;” at the end
- “prepare” the source code for the compiler

Here:

- stdio.h includes standard input/output library functions
- <name.h> vs. “name.h”

Hello World! (IV)

```
/* printing program */
```

```
#include <stdio.h>
```

```
int main (void)
```

```
{
```

```
    printf ("Hello\n World!\n");  
    return 0;
```

```
}
```

```
// int main () also works
```

```
/* that's correct */
```

```
// classic example
```

- "int main (void){ ...}" - program starts here!
- all programs have one main function
- (almost) everything happens inside functions
- function header and function body {...} consisting of statements and expressions

Hello World!

```
/* printing program */  
#include <stdio.h>  
  
int main (void) // int main () also works  
{  
    printf ("Hello\n World!\n"); /* that's correct */  
    return 0;  
} // classic example
```

Statements:

- call function `printf("...")` – prints series of characters on standard output (in this case on the screen)
- `"\n"` is an escape sequence (see table in VtC)
- main function returns status code `"0"` and ends the program

C: "Hello World!"

```
/* printing program */  
#include <stdio.h>  
  
int main (void) //  
{  
    printf ("Hello\n World!\n");    /* that's correct */  
    printf("\nEnter press to exit...\n");  
    getchar();  
    return 0;  
}  
// classic example
```

← “Run the program until enter is pressed”

One way to ensure that the window does not close right after execution of the program: use *getchar()*

Summary (I)

- Computers: Central Processing Unit (CPU) + Memory + Input/Output (I/O)
 - Program and data are stored in memory
 - Computer program: set of instructions to manipulate data
 - Compiling + linking gives an executable
-

Summary (II)

- Structured programming: proper algorithm, implement, comment, test, debug, debug, debug...
 - History of C – related to the history of UNIX/LINUX
 - Hello World! – a first program
-